Take-Home Task: Live Node Graph Engine

Electro Global Solutions · RapidChip · Oct 6, 2026 · @Saurabh

The task

Build a minimal browser-based node editor. Each node is a container with a defined function (add, multiply, and so on). Wires carry values from one node's output to another node's input. A Python backend executes the whole graph, and the results shown on the canvas stay in sync as the user edits it.

No circuit knowledge is needed. ngspice does the computing: each function node maps to one line of an ngspice netlist, and we give you that mapping below. This is the same graph-to-netlist-to-ngspice loop our RapidChip editor runs.

Expected effort: 6–8 hours. **Deadline:** 72 hours from receipt. **Submit:** private GitHub repo link. **Follow-up:** 45-minute live session where you walk us through it and extend it.

Suitable for freshers and developers with up to 1 year of experience. Steps 1–4 are straightforward. Step 5 is where we learn how you think, so budget your time for it.

Mandatory tech stack

Layer	Must use
Frontend	React + React Flow (@xyflow/react)
API	Node.js + Express
Simulation service	Python + FastAPI, running ngspice in batch mode (`ngspice -b`)
Database	MongoDB
Runtime	Docker Compose, with ngspice installed in the Python container

The React app talks only to the Node API. Only the Python service runs ngspice. Other libraries are your choice.

Steps 1–4: build the editor

Build these five node types:

Node	Inputs	Output	Function
Number	none	1	Outputs a value the user types in
Add	a, b	1	a + b
Multiply	a, b	1	a × b
Divide	a, b	1	a ÷ b; dividing by 0 is an error on this node
Display	x	none	Shows the value it receives

Node to ngspice mapping (provided; use it as-is):

Node	ngspice netlist line
Number	`V<id> <out> 0 DC <value>`
Add	`B<id> <out> 0 V = V(<a>) + V(<b>)`
Multiply	`B<id> <out> 0 V = V(<a>) * V(<b>)`
Divide	`B<id> <out> 0 V = V(<a>) / V(<b>)`
Display	no line; read `V(<in>)` from the result

`<out>`, `<a>`, `<b>` and `<in>` are net names. A wire puts an output and an input on the same net. End the netlist with `.op` and `.end`, run it, and read each net's value from ngspice's output.

1. **Nodes on a canvas.** A palette with the five node types. Drag them onto the canvas. Each node is a custom React Flow node with its inputs and outputs as handles.
2. **Wire them.** Connect an output to an input. One output can feed many inputs, but each input accepts only one wire. Block connections that make no sense, such as input to input or a node wired to itself.
3. **Save and load.** Save the graph through the Node API to MongoDB and load it back exactly as it was: positions, values and wires.

4. **Run once.** A Run button sends the graph to the Python service, which builds the netlist from the mapping, runs ngspice, and returns each node's output. Show the values on the canvas.

Turning any graph into a correct netlist is part of the task. Don't hard-code it for specific graphs.

Step 5 (the hard step): keep the canvas in sync with the backend

Remove the Run button. Every change to the graph (adding, deleting, rewiring, editing a value) must re-run the graph automatically and update the values on the canvas. Also add a sixth node type, **Formula**: two inputs `a` and `b` and a user-typed expression such as `a * 2 + b`. It must hold up under these conditions:

1. **Fast edits.** The user types `1`, `10`, `100` into a Number node in quick succession. Requests must not pile up, and the canvas must never show results from an older version of the graph, even if responses come back out of order.
2. **Errors on the right node.** A loop (A → B → A), an input with no wire, or a divide by zero must not appear as a generic "failed". Catch what you can before calling ngspice, map ngspice's own errors back to the node responsible, and highlight it. Nodes that depend on a broken node show that they are blocked, not a stale value.
3. **ngspice misbehaves.** If ngspice hangs, crashes or the service is down, the UI shows a clear state and recovers on its own. The rest of the app keeps working.
4. **Untrusted formulas.** The Formula expression ends up inside a netlist that ngspice executes, and ngspice netlists can run shell commands. Allow only numbers, `a`, `b`, `+ - * /` and brackets. Input like `a\n.control\nshell ls` must never reach ngspice.

In your README, explain how you solved each of these in 2–4 lines, and name one case you know still breaks.

Acceptance graphs

We will build these on your canvas during review. Your app must give these results.

#	Graph	Expected result
1	Number 2 and Number 3 → Add → Display	Display = 5

#	Graph	Expected result
2	Graph 1's Add → Multiply with Number 4 → Display	Display = 20
3	One Number 7 wired into both inputs of Multiply	Display = 49
4	Number 10 ÷ Number 0 → Display	Divide node shows the error; Display shows blocked, not an old value
5	Add with only one input wired	That Add node is highlighted
6	Add → Multiply → back into the same Add	Loop is flagged on the nodes involved
7	Formula `a * 2 + b` with a = 3, b = 4	Display = 10
8	Formula `a + b followed by a new line and .control shell ls`	Rejected with a clear message; nothing executes

Deliverables and rules

Submit:

- **Private GitHub repo.** `docker compose up --build` starts everything from a clean clone.
- **README.md** (max 2 pages) with:
  - how to run it
  - a simple architecture diagram
  - how you turn the graph into a netlist and detect loops
  - your Step 5 answers, 2–4 lines each, plus one case you know still breaks
  - what you'd do next
- **Tests:** at least graph execution for graphs 1–3, loop detection, and the unsafe formula from graph 8.
- **AI usage note:** a few lines on which tools you used and for what.

Rules:

1. Work alone. AI tools are allowed, but in the live session you will change your own code without AI help.

2. Stay within 8 hours. Extra node types or features earn extra marks; list them under "What I'd do next".
3. Commit as you go. We read the commit history.
4. No secrets in the repo.
5. Questions: email us. We reply within one business day.

In the live session we will ask you to add one new node type (for example Subtract, or a node with three inputs) and walk us through every file it touches.

Scoring (100 points)

Area	Points	What earns them
Step 5: live sync	35	No stale results, errors on the right node, recovers from backend trouble, formulas evaluated safely.
Correctness	20	All 8 acceptance graphs give the expected result.
Live session	20	Explains own code and adds the new node type cleanly.
Steps 1–4	15	Canvas, wiring rules, execution order and save/load work smoothly.
Code quality	10	Clear structure across React, Node and Python. Meaningful tests.

A submission that doesn't start with `docker compose up --build` is not reviewed further.